introduction: Three Ways to Teach an LLM

1. Prompt / System Instructions

This is the simplest method.

You do **not** change the model. You only give it instructions before the question.

Example:

You are answering Dr. Ahmed.

He is working on local LLMs, Flutter, Flask, and Arabic RAG.

Always answer in Arabic.

Use simple technical explanations.

In your Flask code, this is the part called:

system_prompt

or:

profile.txt

How it works

Every time you ask a question, your Flask wrapper sends:

User profile

Rules

Question

to Qwen.

So Qwen behaves as if it knows you, but it does not really store this information inside the model.

Best use

Use this for:

Your name

Your position

Preferred answer style

Answer language

Formatting rules

System behavior

Example:

Always answer in Arabic.

Do not show reasoning.

Answer as a university professor.

Keep answers concise.

Limitation

If you do not send the profile or system prompt, the model forgets it.

So this is:

Temporary behavior control

not real learning.

---

2. RAG — Retrieval-Augmented Generation

This is the best method for your **Arabic novel project**.

RAG means the model does not memorize the whole novel. Instead, the system stores the novel outside the model, searches it, and sends the relevant parts to Qwen when needed.

How it works

Arabic novel text file

↓

Split into chunks

↓

Convert chunks into embeddings

↓

Store vectors in FAISS

↓

User asks a question

↓

Convert question into embedding

↓

Retrieve relevant chunks

↓

Send chunks + question to Qwen

↓

Qwen answers in Arabic

Example

Question:

أين وقعت أحداث القصة؟

FAISS retrieves relevant passages from the novel:

Passage 2

Passage 5

Then Qwen receives:

Question + relevant passages

and answers:

وقعت أحداث القصة في مكتب الإدارة داخل دار نشر...

Best use

Use RAG for:

Arabic novels

Books

PDFs

Course notes

Research papers

Policies

Large documents

Knowledge bases

Why RAG is better for novels

A novel may be too large to put inside one prompt.

RAG solves this by sending only the relevant parts.

Limitation

The model does not permanently learn the novel.

The knowledge is stored in:

arabic_novel.txt
chunks.json
faiss.index
If the retrieval gets the wrong passage, the answer may be wrong.
So RAG quality depends on:
OCR quality
Chunk size
Embedding model
top_k value
Prompt design

---

3. Fine-tuning / LoRA
This is the real training method.
Fine-tuning means you update the model's behavior using training examples.
LoRA means:
Low-Rank Adaptation
It is a lightweight fine-tuning method. Instead of changing the whole model, it trains a small adapter.
How it works
You prepare a dataset like this:

```
{
  "instruction": "Answer this type of question in Arabic.",
  "input": "Where did the story happen?",
  "output": "وقعت أحداث القصة في..."
}
```

Then you train a LoRA adapter on many examples.
The result is:
Base Qwen model
+
LoRA adapter
=
Customized model behavior
Best use
Use LoRA when you want the model to learn:
A repeated answer style
A specific task format
A domain-specific writing style
A classification task
A chatbot personality
A structured response pattern
Not best for
LoRA is **not the best choice** for teaching a full Arabic novel.
Why?

Because the model may not memorize exact details reliably, and training requires many examples.
For novel question-answering, RAG is better.

Comparison Table

Technique	Does it change model weights?	Best for	Storage location	Difficulty
Prompt / System Instructions	No	Rules, style, identity, language	Sent with each request	Easy
RAG	No	Books, novels, PDFs, documents	External text + FAISS index	Medium
Fine-tuning / LoRA	Yes, adapter or weights	New behavior, style, task pattern	LoRA adapter / trained model	Advanced

Technical Notes: Arabic document RAG using Local Qwen3-8B

1. Objective

The objective of this lab is to make a local LLM answer detailed questions about an Arabic document stored as a text file.

The LLM will not be fine-tuned. Instead, we will use **Retrieval-Augmented Generation**.

The system will:

Read Arabic document text file
↓
Clean and split the document into chunks
↓
Create searchable embeddings for each chunk
↓
Store the chunks and embeddings locally
↓
Receive a question from browser or Flutter
↓
Search for the most relevant document chunks
↓
Send the question + retrieved chunks to Qwen3-8B
↓
Return the answer as HTML or JSON

---

2. Why RAG instead of fine-tuning?

Fine-tuning is used when we want to change the model's behavior or style permanently. It requires training data, training code, and usually GPU resources.

For your goal, you want the model to answer questions about a specific document. That is a **knowledge retrieval problem**, so RAG is better.

Requirement	**Best method**
Know your name and role	Profile/system prompt
Remember current conversation	Conversation history
Answer questions from Arabic document	RAG
Permanently change model behavior	Fine-tuning / LoRA [Low-Rank Adaptation]

In this lab, the document stays in a text file, and the system retrieves relevant passages when needed.

---

3. Required folders

Create this structure:

D:\llama\
|
├── pyrun_rag.py
├── build_document_index.py
├── profile.txt

|
├── data\
| └── arabic_document.txt
|
├── rag_index\
| ├── chunks.json
| └── faiss.index

The most important file is:

D:\llama\data\arabic_document.txt

This file should contain the Arabic document text extracted from OCR or typed/exported text.

---

4. Install required Python packages

Run:

pip install flask requests sentence-transformers faiss-cpu numpy

What each package does:

Package	Purpose
flask	Build local web API
requests	Send question to llama-server
sentence-transformers	Convert text chunks/questions into embeddings
faiss-cpu	Fast similarity search over embeddings
numpy	Numeric arrays for vectors

Sentence Transformers provides models for computing embeddings and similarity, and Faiss is designed for efficient similarity search over dense vectors.

---

5. Prepare the Arabic document text file

Save the document as:

D:\llama\data\arabic_document01.txt

Recommended format:

الفصل الأول

كان المكان هادئًا في تلك الليلة...

الفصل الثاني

بدأت الأحداث تتغير عندما...

Try to keep chapter titles if available because they help retrieval.

Good text quality is important. If the text came from OCR, manually check a few pages first.

---

6. Create personal profile file

Create:

D:\llama\profile.txt

Example:

User Profile:
Name: Prof. Ahmed ElShafee.
Position: Professor of electrical Engineering and Computer Science.
Interests: Artificial Intelligence, local LLMs, applied AI education, embedded systems, cloud computing, cybersecurity, and educational technology.
Preferred answer style: Clear, practical, step-by-step, suitable for students and technical labs.
Language preference: English by default, Arabic when requested.
This profile will be sent as a system message with each request.

Part A: Build the Document Index

7. Create build_document_index.py

Create this file:

D:\llama\build_document01_index.py

Paste this code:

```
import os
import json
import re
import numpy as np
import faiss
from sentence_transformers import SentenceTransformer

DOCUMENT_FILE = r"D:\llama\data\arabic_document01.txt"
OUTPUT_DIR = r"D:\llama\rag_index"

CHUNKS_FILE = os.path.join(OUTPUT_DIR, "chunks01.json")
FAISS_INDEX_FILE = os.path.join(OUTPUT_DIR, "faiss01.index")

# Good multilingual embedding model for Arabic + English
EMBEDDING_MODEL_NAME = "sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2"


def clean_arabic_text(text):
    # Remove repeated spaces
    text = re.sub(r"[ \t]+", " ", text)

    # Normalize newlines
    text = re.sub(r"\n{3,}", "\n\n", text)

    # Remove Tatweel
    text = text.replace("-", "")

    return text.strip()
```

```
def split_into_paragraphs(text):
    paragraphs = []

    for part in text.split("\n"):
        part = part.strip()
        if part:
            paragraphs.append(part)

    return paragraphs


def build_chunks(paragraphs, max_chars=1200, overlap_chars=200):
    chunks = []
    current = ""

    for paragraph in paragraphs:
        if len(current) + len(paragraph) + 1 <= max_chars:
            current += "\n" + paragraph
        else:
            if current.strip():
                chunks.append(current.strip())

            # Add small overlap from previous chunk
            overlap = current[-overlap_chars:] if len(current) > overlap_chars else current
            current = overlap + "\n" + paragraph

    if current.strip():
        chunks.append(current.strip())

    return chunks


def main():
    os.makedirs(OUTPUT_DIR, exist_ok=True)

    if not os.path.exists(DOCUMENT_FILE):
        raise FileNotFoundError(f"Document file not found: {DOCUMENT_FILE}")

    print("Reading Arabic document...")
    with open(DOCUMENT_FILE, "r", encoding="utf-8") as f:
        text = f.read()

    text = clean_arabic_text(text)
```

```
    paragraphs = split_into_paragraphs(text)
    chunks = build_chunks(paragraphs)

    print(f"Paragraphs: {len(paragraphs)}")
    print(f"Chunks: {len(chunks)}")

    chunk_records = []
    for i, chunk in enumerate(chunks):
        chunk_records.append({
            "id": i,
            "text": chunk
        })

    print("Loading embedding model...")
    model = SentenceTransformer(EMBEDDING_MODEL_NAME)

    print("Encoding chunks...")
    embeddings = model.encode(
        chunks,
        convert_to_numpy=True,
        show_progress_bar=True,
        normalize_embeddings=True
    )

    embeddings = embeddings.astype("float32")

    dimension = embeddings.shape[1]
    index = faiss.IndexFlatIP(dimension)

    print("Building FAISS index...")
    index.add(embeddings)

    print("Saving files...")
    with open(CHUNKS_FILE, "w", encoding="utf-8") as f:
        json.dump(chunk_records, f, ensure_ascii=False, indent=2)

    faiss.write_index(index, FAISS_INDEX_FILE)

    print("Done.")
    print("Chunks saved to:", CHUNKS_FILE)
    print("Index saved to:", FAISS_INDEX_FILE)


if __name__ == "__main__":
```

```
main()
```

---

8. Run the index builder

Run:

cd D:\llama

python build_document01_index.py

Expected output:

Reading Arabic document...

Paragraphs: 850

Chunks: 120

Loading embedding model...

Encoding chunks...

Building FAISS index...

Saving files...

Done.

After this, you should have:

D:\llama\rag_index\chunks.json

D:\llama\rag_index\faiss.index

This means the document is now searchable.

FAISS : Facebook AI Similarity Search
It is a library used to search quickly inside a large collection of vectors. In your Arabic novel RAG project, FAISS is used to find the most relevant parts of the novel for a user question. **Simple idea** Imagine your Arabic novel is split into many small chunks: Chunk 1: بداية الرواية... Chunk 2: وصف الشخصية الرئيسية... Chunk 3: الحوار بين الشخصيات... Chunk 4: نهاية الفصل الأول... Each chunk is converted into a numerical vector called an **embedding**. When the user asks: من هو بطل الرواية؟ the question is also converted into an embedding. Then FAISS compares the question embedding with all novel chunk embeddings and returns the closest chunks. **In your RAG system** The flow is: Arabic novel text ↓ Split into chunks ↓

Convert chunks into embeddings
↓
Store embeddings in FAISS
↓
User asks a question
↓
Convert question into embedding
↓
FAISS finds the closest chunks
↓
Send those chunks to Qwen3-8B
↓
Qwen answers using the retrieved text

Why FAISS is important

Without FAISS, your program may need to search the whole novel every time.
With FAISS, it can quickly find the most relevant passages.

Example

Question:
لماذا سافر البطل؟
FAISS may retrieve chunks like:
Passage 12: كان سبب سفره هو البحث عن عمل...
Passage 13: ودّع أسرته في الصباح...
Passage 14: وصل إلى المدينة الجديدة...
Then Qwen receives only these passages and answers based on them.

Simple explanation for students

FAISS is the search engine inside the RAG system.
Or:
FAISS helps the LLM find the right pages before answering.

Part B: Flask RAG Wrapper

9. Create pyrun_rag.py

Create this file:

D:\llama\pyrun_rag01.py
Paste this code:

```
from flask import Flask, request, jsonify
import requests
import html
import os
import json
import re
import faiss
from sentence_transformers import SentenceTransformer
```

```
app = Flask(__name__)

# ============================================================
# Configuration
# ============================================================

LLAMA_SERVER_URL = "http://127.0.0.1:8080/v1/chat/completions"

PROFILE_FILE = r"D:\llama\profile01.txt"
CHUNKS_FILE = r"D:\llama\rag_index\chunks01.json"
FAISS_INDEX_FILE = r"D:\llama\rag_index\faiss01.index"

EMBEDDING_MODEL_NAME = "sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2"

DEFAULT_TOP_K = 5
MAX_TOP_K = 10

FAST_MAX_TOKENS = 250
NORMAL_MAX_TOKENS = 400
DETAILED_MAX_TOKENS = 700

LLAMA_TIMEOUT = 600

# ============================================================
# Global RAG resources
# ============================================================

embedding_model = None
faiss_index = None
chunks = None

RAG_READY = False
RAG_LOAD_ERROR = ""


# ============================================================
# Basic routes
# ============================================================

@app.route("/favicon.ico")
def favicon():
    return "", 204
```

```
# ============================================================
# Utility functions
# ============================================================

def read_file(path):
    if not os.path.exists(path):
        return ""

    with open(path, "r", encoding="utf-8") as f:
        return f.read()


def load_profile():
    profile = read_file(PROFILE_FILE).strip()

    if not profile:
        profile = (
            "The user is working on local LLM experiments using Qwen3-8B, "
            "llama.cpp, Flask, Flutter, and Arabic RAG."
        )

    return profile


def load_rag_resources():
    """
    Load the embedding model, document chunks, and FAISS index when the server starts.
    This prevents timeout on the first question.
    """
    global embedding_model, faiss_index, chunks, RAG_READY, RAG_LOAD_ERROR

    try:
        print("=" * 70)
        print("Starting Arabic Document RAG Flask Server")
        print("=" * 70)

        if not os.path.exists(CHUNKS_FILE):
            raise FileNotFoundError(
                f"Chunks file not found: {CHUNKS_FILE}. "
                "Run build_document_index.py first."
            )
```

```
        if not os.path.exists(FAISS_INDEX_FILE):
            raise FileNotFoundError(
                f"FAISS index file not found: {FAISS_INDEX_FILE}. "
                "Run build_document_index.py first."
            )

        print("Loading embedding model...")
        embedding_model = SentenceTransformer(EMBEDDING_MODEL_NAME)
        print("Embedding model loaded.")

        print("Loading chunks...")
        with open(CHUNKS_FILE, "r", encoding="utf-8") as f:
            chunks = json.load(f)
        print(f"Chunks loaded: {len(chunks)}")

        print("Loading FAISS index...")
        faiss_index = faiss.read_index(FAISS_INDEX_FILE)
        print("FAISS index loaded.")

        RAG_READY = True
        RAG_LOAD_ERROR = ""

        print("=" * 70)
        print("RAG system is ready.")
        print("Open: http://127.0.0.1:5000")
        print("=" * 70)

    except Exception as e:
        RAG_READY = False
        RAG_LOAD_ERROR = str(e)

        print("=" * 70)
        print("RAG system failed to load.")
        print(RAG_LOAD_ERROR)
        print("=" * 70)


def get_safe_top_k():
    raw_top_k = request.args.get("top_k", str(DEFAULT_TOP_K)).strip()

    try:
        top_k = int(raw_top_k)
    except ValueError:
        top_k = DEFAULT_TOP_K
```

```
    if top_k < 1:
        top_k = 1

    if top_k > MAX_TOP_K:
        top_k = MAX_TOP_K

    return top_k


def get_answer_mode_settings():
    mode = request.args.get("mode", "fast").strip().lower()

    if mode == "detailed":
        return "detailed", DETAILED_MAX_TOKENS

    if mode == "normal":
        return "normal", NORMAL_MAX_TOKENS

    return "fast", FAST_MAX_TOKENS


def contains_arabic(text):
    return bool(re.search(r"[\u0600-\u06FF]", text))


def count_english_words(text):
    return len(re.findall(r"[A-Za-z]{3,}", text))


def count_arabic_words(text):
    return len(re.findall(r"[\u0600-\u06FF]+", text))


def answer_looks_english(answer):
    """
    Detect if the model answered mostly in English.
    If yes, a second request rewrites the answer into Arabic.
    """
    english_count = count_english_words(answer)
    arabic_count = count_arabic_words(answer)

    if english_count > arabic_count:
        return True
```

```
    if not contains_arabic(answer) and english_count > 5:
        return True

    return False


def force_arabic_answer(answer):
    """
    Second-pass correction:
    If Qwen answers in English, rewrite the same answer in Arabic.
    """
    payload = {
        "messages": [
            {
                "role": "system",
                "content": (
                    "أنت مساعد عربي. مهمتك الوحيدة هي إعادة صياغة النص التالي باللغة العربية فقط. "
                    "لا تضف معلومات جديدة. لا تشرح. لا تستخدم الإنجليزية. "
                    "أعد كتابة الإجابة فقط باللغة العربية الفصحى الواضحة."
                )
            },
            {
                "role": "user",
                "content": answer
            }
        ],
        "temperature": 0.1,
        "max_tokens": 400
    }

    response = requests.post(
        LLAMA_SERVER_URL,
        json=payload,
        timeout=LLAMA_TIMEOUT
    )

    response.raise_for_status()
    data = response.json()

    corrected = data["choices"][0]["message"].get("content", "").strip()

    if corrected:
        return corrected
```

```
    return answer


# ============================================================
# HTML UI functions
# ============================================================

def question_form(
    default_question="",
    default_format="html",
    default_top_k=DEFAULT_TOP_K,
    default_mode="fast"
):
    safe_question = html.escape(default_question)

    html_selected = "selected" if default_format == "html" else ""
    json_selected = "selected" if default_format == "json" else ""

    fast_selected = "selected" if default_mode == "fast" else ""
    normal_selected = "selected" if default_mode == "normal" else ""
    detailed_selected = "selected" if default_mode == "detailed" else ""

    return f"""
    <div class="question-card">
      <h3>Ask the Local LLM</h3>

      <form method="get" action="/ask">
        <label for="q"><b>Question</b></label>

        <textarea
          id="q"
          name="q"
          placeholder="Type your question here... Example: من هو د. رفعت؟"
          dir="auto"
          required>{safe_question}</textarea>

        <div class="form-row">
          <div>
            <label for="format"><b>Output format</b></label><br>
            <select id="format" name="format">
              <option value="html" {html_selected}>HTML for Browser</option>
              <option value="json" {json_selected}>JSON for Mobile / Flutter</option>
            </select>
```

```
          </div>

          <div>
            <label for="top_k"><b>Retrieved passages</b></label><br>
            <input
              id="top_k"
              name="top_k"
              type="number"
              value="{default_top_k}"
              min="1"
              max="{MAX_TOP_K}">
          </div>

          <div>
            <label for="mode"><b>Answer mode</b></label><br>
            <select id="mode" name="mode">
              <option value="fast" {fast_selected}>Fast</option>
              <option value="normal" {normal_selected}>Normal</option>
              <option value="detailed" {detailed_selected}>Detailed</option>
            </select>
          </div>
        </div>

        <button type="submit">Ask Local LLM</button>
      </form>

      <div class="examples">
        <b>Quick examples:</b><br>
        <a href="/ask?q=من هو لوسيفر؟&format=html&top_k=5&mode=fast">من هو لوسيفر؟
</a><br>
        <a href="/ask?q=من هو د. رفعت؟&format=html&top_k=5&mode=fast">من هو د.
رفعت؟</a><br>
        <a href="/ask?q=أذكر أهم شخصيات القصة؟&format=html&top_k=5&mode=fast">أذكر أهم
شخصيات القصة؟</a><br>
      </div>
    </div>
    """


def page_template(title, body):
    return f"""
    <html>
      <head>
        <title>{html.escape(title)}</title>
```

```
<meta charset="UTF-8">

<style>
  body {{
    font-family: Arial, Tahoma, sans-serif;
    margin: 40px;
    line-height: 1.6;
    background-color: #f8f9fa;
    direction: ltr;
  }}

  .container {{
    max-width: 1000px;
    margin: auto;
    background: white;
    padding: 25px;
    border-radius: 12px;
    box-shadow: 0 2px 8px rgba(0,0,0,0.1);
  }}

  h2 {{
    color: #1f4e79;
  }}

  textarea {{
    width: 100%;
    height: 120px;
    font-size: 16px;
    padding: 12px;
    border-radius: 8px;
    border: 1px solid #ccc;
    box-sizing: border-box;
    direction: auto;
  }}

  input[type="number"] {{
    width: 120px;
    font-size: 16px;
    padding: 8px;
    border-radius: 6px;
    border: 1px solid #ccc;
  }}

  select {{
```

```
    font-size: 16px;
    padding: 8px;
    border-radius: 6px;
    border: 1px solid #ccc;
}}

button {{
    background-color: #1f75cb;
    color: white;
    border: none;
    padding: 12px 22px;
    font-size: 16px;
    border-radius: 8px;
    cursor: pointer;
    margin-top: 12px;
}}

button:hover {{
    background-color: #155a99;
}}

pre {{
    background: #f2f2f2;
    padding: 12px;
    border-radius: 8px;
    overflow-x: auto;
    direction: ltr;
    text-align: left;
}}

table {{
    border-collapse: collapse;
    width: 100%;
    margin-top: 15px;
    direction: ltr;
}}

th, td {{
    border: 1px solid #ccc;
    padding: 8px;
    text-align: left;
    vertical-align: top;
}}
```

```
th {{
    background-color: #eaf4ff;
}}

.note {{
    background: #fff8df;
    padding: 12px;
    border-radius: 8px;
    border-left: 5px solid #f0c040;
    margin-bottom: 15px;
}}

.question-card {{
    background: #f4f9ff;
    padding: 18px;
    border-radius: 10px;
    border: 1px solid #d6e9ff;
    margin-bottom: 25px;
}}

.form-row {{
    display: flex;
    gap: 20px;
    margin-top: 12px;
    flex-wrap: wrap;
}}

.answer-box {{
    background:#eaf4ff;
    padding:15px;
    border-radius:8px;
    white-space:pre-wrap;
    direction: rtl;
    text-align: right;
    font-size: 17px;
}}

.question-box {{
    background:#f2f2f2;
    padding:15px;
    border-radius:8px;
    direction:auto;
    font-size: 17px;
}}
```

```
            .arabic-preview {{
                direction: rtl;
                text-align: right;
                font-size: 15px;
            }}

            .examples {{
                margin-top: 15px;
                background: white;
                padding: 12px;
                border-radius: 8px;
            }}

            .error {{
                background: #ffecec;
                border-left: 5px solid #d9534f;
                padding: 12px;
                border-radius: 8px;
            }}

            a {{
                color: #155a99;
            }}
        </style>
    </head>

    <body>
        <div class="container">
            {body}
        </div>
    </body>
</html>
"""


# ============================================================
# RAG functions
# ============================================================

def retrieve_chunks(question, top_k=DEFAULT_TOP_K):
    if not RAG_READY:
        raise RuntimeError(
            "RAG resources are not ready. "
```

```
            f"Load error: {RAG_LOAD_ERROR}"
        )

    query_embedding = embedding_model.encode(
        [question],
        convert_to_numpy=True,
        normalize_embeddings=True
    ).astype("float32")

    scores, indices = faiss_index.search(query_embedding, top_k)

    results = []

    for score, idx in zip(scores[0], indices[0]):
        if idx == -1:
            continue

        record = chunks[int(idx)]

        results.append({
            "id": record["id"],
            "score": float(score),
            "text": record["text"]
        })

    return results


def call_llama(question, retrieved_chunks, max_tokens=FAST_MAX_TOKENS,
answer_mode="fast"):
    profile_text = load_profile()

    context_blocks = []

    for item in retrieved_chunks:
        context_blocks.append(
            f"[Passage ID: {item['id']} | Score: {item['score']:.4f}]\n{item['text']}"
        )

    document_context = "\n\n---\n\n".join(context_blocks)

    system_prompt = f"""
You are a local Arabic document question-answering assistant.
```

```
User profile:
{profile_text}

Mandatory rules:
1. The final answer MUST be in Arabic only, even if the question is written in English.
2. Do not answer in English.
3. Use ONLY the provided Arabic document passages.
4. If the answer is not clearly found in the passages, say exactly:
  "لا أستطيع تحديد الإجابة من النصوص المتاحة."
5. Do not invent events, characters, places, relationships, or explanations.
6. Do not repeat the full retrieved passages.
7. Keep the answer concise and clear.
8. Mention passage IDs only if useful.
9. Give the final answer only. Do not show reasoning.
10. If an English technical term is unavoidable, keep it minimal; the explanation must remain
Arabic.

Answer mode:
{answer_mode}

Arabic document passages:
{document_context}
"""

   payload = {
      "messages": [
         {
            "role": "system",
            "content": system_prompt
         },
         {
            "role": "user",
            "content": (
               "Answer the following question in Arabic only using the provided passages:\n\n"
               + question
            )
         }
      ],
      "temperature": 0.1,
      "max_tokens": max_tokens
   }

   response = requests.post(
      LLAMA_SERVER_URL,
```

```
    json=payload,
    timeout=LLAMA_TIMEOUT
)

response.raise_for_status()
data = response.json()

message = data["choices"][0]["message"]
answer = message.get("content", "").strip()

if not answer:
    answer = (
        "لم يرجع النموذج إجابة نهائية. "
        "جرّب تقليل عدد المقاطع أو زيادة المهلة أو تعطيل reasoning."
    )

if answer_looks_english(answer):
    answer = force_arabic_answer(answer)

timings = data.get("timings", {})
usage = data.get("usage", {})

return {
    "question": question,
    "answer": answer,
    "model": data.get("model", ""),
    "answer_language": "Arabic only",
    "answer_mode": answer_mode,
    "retrieved_passages": [
        {
            "id": item["id"],
            "score": item["score"],
            "preview": item["text"][:300]
        }
        for item in retrieved_chunks
    ],
    "prompt_tokens": usage.get("prompt_tokens"),
    "completion_tokens": usage.get("completion_tokens"),
    "total_tokens": usage.get("total_tokens"),
    "prompt_tokens_per_second": timings.get("prompt_per_second"),
    "generation_tokens_per_second": timings.get("predicted_per_second")
}
```

```
# ============================================================
# Pages
# ============================================================

def help_screen():
    if not RAG_READY:
        body = f"""
        <h2>RAG System Not Ready</h2>

        <div class="error">
            <h3>The index was not loaded</h3>
            <p>The server started, but the embedding model or FAISS index was not loaded.</p>
            <p><b>Error:</b></p>
            <pre>{html.escape(RAG_LOAD_ERROR)}</pre>
        </div>

        <h3>How to fix</h3>
        <p>Make sure these files exist:</p>
        <pre>{html.escape(CHUNKS_FILE)}
{html.escape(FAISS_INDEX_FILE)}</pre>

        <p>If they do not exist, run:</p>
        <pre>python build_document_index.py</pre>

        <p>Then restart the server:</p>
        <pre>python pyrun_rag.py</pre>
        """

        return page_template("RAG System Not Ready", body)

    body = f"""
    <h2>Arabic Document RAG Assistant</h2>

    <div class="note">
        <b>Status:</b> RAG index is loaded and ready.<br>
        <b>Loaded chunks:</b> {len(chunks)}<br>
        <b>Embedding model:</b> {html.escape(EMBEDDING_MODEL_NAME)}<br>
        <b>Interface language:</b> English<br>
        <b>Answer language:</b> Arabic only
    </div>

    <p>
        This server receives a question, retrieves relevant passages from the Arabic document,
        sends them to the local Qwen3-8B model, and returns the answer in Arabic.
```

```
    </p>

    {question_form(default_format="html", default_top_k=DEFAULT_TOP_K)}

    <h3>Manual browser example</h3>
    <pre>http://127.0.0.1:5000/ask?q= من هو د.
رفعت؟&format=html&top_k=3&mode=fast</pre>

    <h3>Manual mobile / Flutter example</h3>
    <pre>http://127.0.0.1:5000/ask?q= من هو د.
رفعت؟&format=json&top_k=3&mode=fast</pre>

    <h3>Required services</h3>

    <p>Run llama-server first:</p>

    <pre>llama-server.exe -m "D:\\llama\\models\\Qwen_Qwen3-8B-Q4_K_M.gguf" --host
127.0.0.1 --port 8080 --ctx-size 4096 --threads 8</pre>

    <p>Then run this Flask server:</p>

    <pre>python pyrun_rag.py</pre>
    """

    return page_template("Arabic Document RAG Assistant", body)


@app.get("/")
def home():
    return help_screen()


@app.get("/status")
def status():
    if RAG_READY:
        return jsonify({
            "ready": True,
            "chunks": len(chunks),
            "embedding_model": EMBEDDING_MODEL_NAME,
            "interface_language": "English",
            "answer_language": "Arabic only"
        })

    return jsonify({
```

```
        "ready": False,
        "error": RAG_LOAD_ERROR
    }), 503


@app.get("/ask")
def ask():
    question = request.args.get("q", "").strip()
    output_format = request.args.get("format", "html").strip().lower()
    top_k = get_safe_top_k()
    answer_mode, max_tokens = get_answer_mode_settings()

    if output_format not in ["html", "json"]:
        output_format = "html"

    if not question:
        if output_format == "json":
            return jsonify({
                "message": "No question provided.",
                "usage": "/ask?q=Your question here&format=json&top_k=3&mode=fast",
                "ready": RAG_READY
            }), 400

        return help_screen(), 200

    if not RAG_READY:
        error = {
            "error": "RAG system is not ready.",
            "details": RAG_LOAD_ERROR,
            "solution": "Run build_document_index.py, then restart pyrun_rag.py."
        }

        if output_format == "json":
            return jsonify(error), 503

        body = f"""
        <h2>RAG System Not Ready</h2>

        <div class="error">
            <pre>{html.escape(RAG_LOAD_ERROR)}</pre>
        </div>

        <p>Run:</p>
        <pre>python build_document_index.py</pre>
```

```
    <p>Then restart:</p>
    <pre>python pyrun_rag.py</pre>

    <br>
    <a href="/">Back to Home</a>
    """

    return page_template("RAG System Not Ready", body), 503

try:
    retrieved = retrieve_chunks(question, top_k=top_k)

    result = call_llama(
        question,
        retrieved,
        max_tokens=max_tokens,
        answer_mode=answer_mode
    )

    if output_format == "json":
        return jsonify(result)

    safe_question = html.escape(result["question"])
    safe_answer = html.escape(result["answer"])

    passage_rows = ""
    for p in result["retrieved_passages"]:
        passage_rows += f"""
        <tr>
            <td>{p["id"]}</td>
            <td>{p["score"]:.4f}</td>
            <td class="arabic-preview">{html.escape(p["preview"])}</td>
        </tr>
        """

    body = f"""
    <h2>Arabic Document RAG Answer</h2>

    {question_form(
        default_question="",
        default_format="html",
        default_top_k=top_k,
        default_mode=answer_mode
```

```
        )}

        <h3>Question</h3>
        <div class="question-box">
          {safe_question}
        </div>

        <h3>Answer</h3>
        <div class="answer-box">
          {safe_answer}
        </div>

        <h3>Retrieved Passages</h3>
        <table>
          <tr>
            <th>Passage ID</th>
            <th>Similarity Score</th>
            <th>Preview</th>
          </tr>
          {passage_rows}
        </table>

        <h3>Model Information</h3>
        <table>
          <tr><td>Model</td><td>{html.escape(str(result["model"]))}</td></tr>
          <tr><td>Answer mode</td><td>{html.escape(str(result["answer_mode"]))}</td></tr>
          <tr><td>Answer language</td><td>Arabic only</td></tr>
          <tr><td>Prompt tokens</td><td>{result["prompt_tokens"]}</td></tr>
          <tr><td>Completion tokens</td><td>{result["completion_tokens"]}</td></tr>
          <tr><td>Total tokens</td><td>{result["total_tokens"]}</td></tr>
          <tr><td>Prompt speed</td><td>{result["prompt_tokens_per_second"]}
tokens/sec</td></tr>
          <tr><td>Generation speed</td><td>{result["generation_tokens_per_second"]}
tokens/sec</td></tr>
        </table>

        <br>
        <a href="/">Back to Home</a>
        """

        return page_template("Arabic Document RAG Answer", body)

    except requests.exceptions.ConnectionError:
        error = {
```

```
            "error": "Cannot connect to llama-server.",
            "details": "Make sure llama-server is running on http://127.0.0.1:8080",
            "start_command": 'llama-server.exe -m "D:\\llama\\models\\Qwen_Qwen3-8B-
Q4_K_M.gguf" --host 127.0.0.1 --port 8080 --ctx-size 4096 --threads 8'
        }

        if output_format == "json":
            return jsonify(error), 503

        body = f"""
        <h2>Cannot Connect to llama-server</h2>

        <div class="error">
            <p>{html.escape(error["details"])}</p>
        </div>

        <p>Start llama-server using:</p>
        <pre>{html.escape(error["start_command"])}</pre>

        {question_form(
            default_question=question,
            default_format="html",
            default_top_k=top_k,
            default_mode=answer_mode
        )}

        <br>
        <a href="/">Back to Home</a>
        """

        return page_template("Connection Error", body), 503

    except requests.exceptions.Timeout:
        error = {
            "error": "Timeout.",
            "details": "Try reducing retrieved passages, using Fast mode, or shortening the
question."
        }

        if output_format == "json":
            return jsonify(error), 504

        body = f"""
        <h2>Timeout</h2>
```

```
    <div class="error">
      <p>{html.escape(error["details"])}</p>
    </div>

    {question_form(
      default_question=question,
      default_format="html",
      default_top_k=top_k,
      default_mode="fast"
    )}

    <br>
    <a href="/">Back to Home</a>
    """

    return page_template("Timeout", body), 504

  except Exception as e:
    error = {
      "error": "Unexpected error.",
      "details": str(e)
    }

    if output_format == "json":
      return jsonify(error), 500

    body = f"""
    <h2>Unexpected Error</h2>
    <pre>{html.escape(str(e))}</pre>

    {question_form(
      default_question=question,
      default_format="html",
      default_top_k=top_k,
      default_mode=answer_mode
    )}

    <br>
    <a href="/">Back to Home</a>
    """

    return page_template("Unexpected Error", body), 500
```

```
# ============================================================
# Main entry point
# ============================================================

if __name__ == "__main__":
    load_rag_resources()
    app.run(host="0.0.0.0", port=5000)
```

Part C: Run and Test

10. Run llama-server

Open CMD window 1:

```
cd D:\llama
llama-server.exe -m "D:\llama\models\Qwen_Qwen3-8B-Q4_K_M.gguf" --host 0.0.0.0 --port 8080

cd D:\llama
llama-server.exe -m "D:\llama\models\Qwen_Qwen3-8B-Q4_K_M.gguf" --host 127.0.0.1 --port 8080 --ctx-size 4096 --threads 8 --reasoning off

cd D:\llama
llama-server.exe -m "D:\llama\models\Qwen_Qwen3-8B-Q4_K_M.gguf" --host 127.0.0.1 --port 8080 --ctx-size 4096 --threads 8 --n-gpu-layers all --reasoning off
```

Keep it running.

---

11. Run the RAG Flask server

Open CMD window 2:

```
cd D:\llama
python pyrun_rag01.py
```

You should see:

Running on http://127.0.0.1:5000

Running on http://192.168.100.39:5000

---

12. Test from browser

Open:

http://127.0.0.1:5000/

Then test:

http://127.0.0.1:5000/ask?q=من هو بطل الرواية؟&format=html

Another test:

http://127.0.0.1:5000/ask?q=ما الحدث الرئيسي في بداية الرواية؟&format=html

Test JSON for Flutter:

http://127.0.0.1:5000/ask?q=من هو بطل الرواية؟&format=json

From mobile phone on same Wi-Fi:
http://192.168.100.39:5000/ask?q=من هو بطل الرواية؟&format=json

---

Part D: How to test what the LLM "learned"

13. Test types

Use different levels of questions.

A. Direct fact questions

من هو بطل الرواية؟
ما اسم المدينة التي تدور فيها الأحداث؟
من هي الشخصية التي قابلت البطل في الفصل الأول؟
These test whether retrieval can find simple facts.

B. Event questions

ماذا حدث في بداية الرواية؟
ما السبب الذي دفع الشخصية الرئيسية إلى السفر؟
ما المشكلة الأساسية التي واجهت البطل؟
These test whether the system can connect events.

C. Character relationship questions

ما العلاقة بين الشخصية الأولى والشخصية الثانية؟
هل كانت العلاقة بين البطل ووالده جيدة؟ اذكر الدليل من النص.
These test whether the retrieved passages contain enough evidence.

D. Evidence-based questions

اذكر الدليل من النص على أن البطل كان يشعر بالخوف.
ما الجملة أو الموقف الذي يوضح تغير شخصية البطل؟
These are useful because they force the model to rely on retrieved text.

E. Questions outside the document

ما اسم مؤلف الرواية؟
If the author name is not in the text, the expected answer should be:
لا أستطيع تحديد الإجابة من النصوص المتاحة.
This tests whether the model avoids hallucination.

---

14. Evaluation table

Use this table in your lab report:

Test question	Expected answer	Retrieved passages relevant?	Model answer correct?	Notes
من هو بطل الرواية؟	Character name	Yes / No	Yes / No	—
ماذا حدث في بداية الرواية؟	Main opening event	Yes / No	Yes / No	—
ما العلاقة بين س و ص؟	Relationship	Yes / No	Yes / No	—

Test question	Expected answer	Retrieved passages relevant?	Model answer correct?	Notes
سؤال غير موجود في النص	Should say not available	Yes / No	Yes / No	Hallucination test

---

15. Important parameters

top_k

Controls how many chunks are retrieved.

Example:

top_k=3

or:

top_k=7

Usage:

http://127.0.0.1:5000/ask?q=من هو بطل الرواية؟&format=html&top_k=7

Recommended:

Value	Use
3	Fast, focused
5	Good default
8	More context
10	More complete but slower

Chunk size

In build_document_index.py:

max_chars=1200

overlap_chars=200

Recommended values:

Chunk size	Effect
700 chars	More precise retrieval, less context
1200 chars	Good balance
2000 chars	More context, less precise

For Arabic documents, start with:

max_chars=1200

overlap_chars=200

Temperature

In pyrun_rag.py:

"temperature": 0.2

For RAG, use low temperature because you want factual answers, not creative writing.

Recommended:

Temperature	Use
0.1–0.2	Strict factual QA

Temperature	Use
0.3–0.4	Explanation with some flexibility
0.7+	Creative writing, not recommended for factual RAG

---

16. Common problems

Problem 1: The model gives wrong answer

Possible reasons:
The OCR text has errors
The relevant passage was not retrieved
Chunk size is too small or too large
top_k is too low
The question uses different wording from the document
Try:
Increase top_k to 8
Use a clearer question
Improve OCR text
Use chapter names in the text

Problem 2: The answer says not found

Check the retrieved passage table in the HTML page.
If the correct passage is not shown, the retrieval failed.
Try:
top_k=10
Example:
http://127.0.0.1:5000/ask?q=من هو بطل الرواية؟&format=html&top_k=10

Problem 3: The model invents details

Make the system prompt stricter:
Do not invent any answer.
If the answer is not directly available in the provided passages, say:
"لا أستطيع تحديد الإجابة من النصوص المتاحة."
Also reduce temperature:
"temperature": 0.1

Problem 4: Arabic OCR text is noisy

Clean the text manually or with Python.
Common Arabic OCR errors:
ا / أ / إ confusion
ة / ه confusion
ى / ي confusion
missing punctuation
wrong line breaks
broken words

RAG can work with some OCR noise, but severe OCR errors reduce retrieval quality.

---

17. What the model actually learned

Technically, Qwen did **not** permanently learn the document.
The knowledge is stored in:
D:\llama\data\arabic_document.txt
D:\llama\rag_index\chunks.json
D:\llama\rag_index\faiss.index
At question time, the system gives Qwen the relevant passages.
So the system behaves as if Qwen learned the document, but the model file itself did not change.
This is better for your lab because:
No training required
Works on normal laptop
Easy to update the document
Easy to inspect retrieved passages
More reliable than asking from memory

---

18. Recommended GitHub lab name

Add this as a new lab in your repo:
lab-08-arabic-document-rag
Suggested files:

```
lab-08-arabic-document-rag/
│
├── README.md
├── build_document_index.py
├── pyrun_rag.py
├── sample_questions.md
├── evaluation_table.md
└── notes.md
```

Do not upload a copyrighted full document to GitHub. Use a short public-domain Arabic text sample or your own sample text.

---

19. Final system architecture

Arabic Document TXT
↓
build_document_index.py
↓
Chunks + FAISS index
↓
User asks question from browser or Flutter
↓
Flask RAG wrapper

↓

Embedding search retrieves relevant passages

↓

Retrieved passages + question sent to Qwen3-8B

↓

llama-server generates answer

↓

HTML for browser or JSON for Flutter

This is the correct technical path to make your local Qwen3-8B answer detailed Arabic document questions from a text file.